

RFC 9635 (GNAP) vs GAuth (RFC 0111/0115) - Comprehensive Comparison

RFC 9635 (GNAP) vs GAuth (RFC 0111/0115) - Comprehensive Comparison

Executive Summary

RFC 9635 is an IETF Standards Track specification (October 2024) that defines the Grant Negotiation and Authorization Protocol (GNAP) - a next-generation authorization framework designed to replace OAuth 2.0.

GAuth is a Gimel Foundation specification (RFC 0111/0115, August 2025) designed as a comprehensive authorization framework specifically for AI governance, Power of Attorney delegation, and legal compliance.

Key Finding: GNAP is a **modern successor to OAuth 2.0** focused on flexible, negotiated authorization, while GAuth is a **specialized framework for AI systems** that builds on OAuth 2.0 with added legal/compliance layers. They address related but distinct problem spaces.

1. High-Level Comparison

Aspect	RFC 9635 (GNAP)	GAuth (RFC 0111/0115)
Standards Body	IETF (Internet Engineering Task Force)	Gimel Foundation
Publication Date	October 2024	August 2025
Primary Goal	Modern, flexible authorization protocol to replace OAuth 2.0	AI authorization framework with Power of Attorney
Built On	New protocol (not OAuth 2.0 extension)	OAuth 2.0, OpenID Connect, MCP
Design Philosophy	Grant negotiation, flexibility, state management	Legal delegation chains, AI governance
Interaction Model	Multiple interaction modes (redirect, user_code, app, push)	Subscription + Request-specific flows
Token Model	Access tokens with negotiated rights	Extended Tokens with embedded PoA
State Management	Stateful (Processing → Pending → Approved → Finalized)	Stateful (Subscription → Request flows)
Delegation Model	RO → Client delegation	Multi-party chains (Authorizer → Owner → AI)
Client Authentication	Key proofing (HTTP Sig, MTLS, JWS)	Same + Multi-signature support
Compliance Focus	Technical interoperability	Legal/regulatory (GDPR, HIPAA, PoA laws)

2. Detailed Comparison

2.1 Core Architecture

GNAP (RFC 9635) Roles: - **Authorization Server (AS):** Grants delegated privileges - **Client Instance:** Application consuming resources - **Resource Server (RS):** API provider - **Resource Owner (RO):** Entity granting access - **End User:** Person operating client

Grant Request State Machine:

```
Processing → Pending → Approved → Finalized
      ↓           ↓           ↓
    [Denied] [Timeout] [Revoked]
```

Key Features: - Unique key per client instance (not client_id) - Grant negotiation (client can update requests) - Multiple interaction modes - Continuation access tokens - Token rotation and management API

GAAuth (RFC 0111/0115) Roles (P*P Architecture): - **PEP** (Power Enforcement Point): Supply & demand side - **PDP** (Power Decision Point): Authorization grants - **PIP** (Power Information Point): Authorization server - **PAP** (Power Administration Point): Owner's authorizer - **PVP** (Power Verification Point): Identity verification - **Client:** AI agent or application - **Client Owner:** Legal entity controlling client - **Owner's Authorizer:** Statutory authority holder

Flow Structure:

Subscription (Steps I-VIII)

↓

Request (Steps a-i)

Key Features: - Mandatory subscription setup - Authorization chain validation - Commercial register integration - Extended tokens with PoA metadata - Compliance attestation & tracking

2.2 Authorization Flow

GNAP Authorization Flow

```
Client → AS (grant request)
      ↓
AS determines interaction needed
      ↓
Interaction (redirect/user_code/app/push)
      ↓
Continuation request
      ↓
Access token issued
```

Example Grant Request:

```
{
  "access_token": {
    "access": ["photo-api", "dolphin-metadata"]
  },
}
```

```

"client": {
  "key": {
    "proof": "httpsig",
    "jwk": {...}
  }
},
"interact": {
  "start": ["redirect"],
  "finish": {
    "method": "redirect",
    "uri": "https://client.example/return",
    "nonce": "LKLTi25DK82FX4T4"
  }
}
}

```

Example Grant Response:

```

{
  "continue": {
    "uri": "https://as.example/continue",
    "access_token": {"value": "80UPRY5NM330"},
    "wait": 60
  },
  "interact": {
    "redirect": "https://as.example/interact/4CF492ML",
    "finish": "MBDOFXG4Y5CVJXC821LH"
  }
}

```

GAAuth Authorization Flow

Subscription Setup (One-time)

- ↓
- I. Owner's Authorizer Identity
- II. Client Owner Identity
- III. Authorization Chain
- IV-VIII. Resource Owner Auth
- ↓

Per-Request Flow

- ↓
- a. Request compliance validation
- b. Grant authorization
- c-i. Extended token issuance

Example GAAuth Request:

```

{
  "subscription_id": "sub_abc123",
  "requested_scope": {
    "actions": ["sign_contract", "transfer_funds"],
    "value_limit": 10000
  },
  "poa_credential_ref": "poa_xyz789"
}

```

Example GAAuth Response:

```

{
  "extended_token": {
    "access_token": "gauth_at_...",
    "power_of_attorney": {
      "issuer": "Owner's Authorizer",
      "grantee": "Client AI"
    },
    "authorization_chain": [
      {"entity": "Owner's Authorizer", "verified": true},
      {"entity": "Client Owner", "verified": true}
    ],
    "verification_proof": {
      "pvp_identity_check": true,
      "commercial_register_verified": true
    }
  }
}

```

2.3 Token Model

GNAP Access Tokens Token Attributes:

```

{
  "value": "OS9M2PMHKUR64TB8N6BW7OZB8CDFONP219RP1LT0",
  "flags": ["bearer", "durable"],
  "access": ["photo-api", "dolphin-metadata"],
  "expires_in": 3600,
  "manage": {
    "uri": "https://as.example/token/PRY5NM330",
    "access_token": {"value": "B8CDFONP21"}
  },
  "key": {...}    // Optional: different from client key
}

```

Key Features: - **Bearer or Bound:** Can be bearer or key-bound - **Durable Flag:** Hint for token rotation behavior - **Management API:** Rotate/revoke tokens - **Multiple Tokens:** Can request multiple labeled tokens - **Negotiable Rights:** AS can modify requested access

GAAuth Extended Tokens

```

{
  "access_token": "gauth_at_...",
  "token_type": "GAAuth-Extended",
  "power_of_attorney": {
    "poa_id": "poa_xyz789",
    "issuer": "Owner's Authorizer",
    "grantee": "Client AI",
    "scope": {...},
    "restrictions": {"value_limit": 10000},
    "revocation_status": "active"
  },
  "authorization_chain": [
    {"entity": "Owner's Authorizer", "authority": "Statutory"},
    {"entity": "Client Owner", "authority": "Delegated"},
  ]
}

```

```

    {"entity": "Client AI", "authority": "Granted"}
  ],
  "verification_proof": {
    "pvp_identity_check": true,
    "commercial_register_verified": true
  },
  "compliance_level": "rfc-0111-compliant",
  "legal_framework": "GDPR|HIPAA"
}

```

Key Features: - **Self-Contained:** All authorization info embedded - **PoA Embedded:** Power of Attorney in token - **Authorization Chain:** Full delegation chain - **Compliance Tracking:** Legal framework metadata - **Always Bound:** Never bearer tokens

2.4 Client Instance Identification

GNAP By Value:

```

{
  "client": {
    "key": {
      "proof": "httpsig",
      "jwk": {...}
    },
    "class_id": "web-server-1234",
    "display": {
      "name": "My Client App",
      "uri": "https://client.example"
    }
  }
}

```

By Reference:

```

{
  "client": "client-541-ab"
}

```

Key Principles: - **Instance-Specific Keys:** Each instance has unique key - **No client_id:** Uses key as identifier - **Dynamic Registration:** AS can return instance_id - **Key Rotation:** Supports rotation

GAuth Client Identification:

```

{
  "client_id": "ai_agent_12345",
  "client_owner_id": "company_xyz",
  "owners_authorizer_id": "board_directors"
}

```

Key Principles: - **Multi-Party IDs:** Client, Owner, Authorizer all identified - **Commercial Register:** verificação via official records - **Key Binding:** Multi-signature support - **Static Registration:** Subscription-based

2.5 Interaction Modes

GNAP Interaction Start Methods

1. **redirect**: Redirect to arbitrary URI
2. **app**: Launch application URI
3. **user_code**: Display short user code (static URI)
4. **user_code_uri**: Display code + dynamic URI

Example:

```
{
  "interact": {
    "start": ["redirect", "user_code"],
    "finish": {
      "method": "redirect",
      "uri": "https://client.example/callback",
      "nonce": "LKLTi25DK82FX4T4"
    }
  }
}
```

Interaction Finish Methods: 1. **redirect**: Browser redirect to callback 2. **push**: Direct HTTP POST to callback

Key Features: - **Negotiated**: Client proposes, AS responds - **Multiple Options**: Client can support multiple modes - **Interaction Hash**: Cryptographic binding (hash = BASE64URL(SHA-256(nonce + finish_nonce + interact_ref + AS_URI))) - **One-Time Use**: Interaction references expire after use

GAUTH Interaction Subscription Phase (One-Time): - Identity verification (PVP) - Authorization chain setup - Resource owner authorization

Request Phase (Per-Transaction): - Compliance validation - Grant authorization (if needed) - Token issuance

Key Features: - **Front-Loaded**: Most interaction in subscription - **Policy-Based**: Can be automated for repeat requests - **Asynchronous**: RO authorization can be async - **Compliance-Driven**: Legal requirements drive interaction

2.6 Key Proofing Mechanisms

GNAP Supported Methods: 1. **HTTP Message Signatures** ([RFC9421]) 2. **Mutual TLS** (MTLS) 3. **Detached JWS** 4. **Attached JWS**

HTTP Signature Example:

```
POST /gnap HTTP/1.1
Host: as.example
Content-Type: application/json
Signature-Input: sig1=...
Signature: sig1=...
```

```
{"client": {...}}
```

Key Features: - **Flexible**: Multiple proofing methods - **Per-Request**: Every request signed - **Key Formats**: JWK, JWKS URI, certificate - **Asymmetric Preferred**: Symmetric keys discouraged

GAUTH Proofing Methods: - HTTP Signature (same as G NAP) - MTLS - **Multi-Signature:** Threshold signatures for multi-party - **BLS Aggregated Signatures:** For efficiency

Additional Security: - **Merkle Trees:** Revocation transparency - **Replay Protection:** BoltDB-based nonce tracking - **Timestamp Authority:** External anchoring

3. Major Differences

3.1 Problem Space

GNAP	GAUTH
General-purpose authorization Replace OAuth 2.0 limitations Web apps, APIs, IoT, devices Flexible for any use case	AI-specific authorization Regulate AI agents with legal authority AI systems in regulated industries Specialized for compliance & PoA

3.2 Grant Lifecycle

GNAP

- **Dynamic:** Grants can be updated, modified, extended
- **Negotiation:** Client and AS negotiate access rights
- **Continuation:** Single grant continues across interactions
- **State Transitions:** Complex state machine

GAUTH

- **Subscription-Based:** Upfront relationship establishment
- **Fixed Scope:** Scope defined in subscription
- **Per-Request:** New token per transaction
- **Compliance-Driven:** State tied to compliance validation

3.3 Delegation Model

GNAP

End User → Client Instance

→ AS

→ RO

→ Grants access to client

Single-Level Delegation: RO delegates to client

GAUTH

Owner's Authorizer (Statutory Authority)

↓

Client Owner (Corporate Entity)

↓

Client (AI System)

↓

Resource Owner (Data Subject)

↓
Resource Server validates chain
Multi-Level Delegation: Chain of authority validated

3.4 Token Introspection

GNAP

- **No standard introspection in core spec**
- Self-contained tokens preferred
- Resource Server Connections ([RFC 9767] - if ever published) would add introspection
- Token management API (rotate/revoke)

GAuth

- **Self-contained validation** primary method
- Optional compliance tracking via AS
- Revocation checking
- Authorization chain verification

4. Similarities

Despite their differences, GNAP and GAuth share several concepts:

Feature	GNAP	GAuth
Stateful Protocol	Explicit state machine	Subscription + Request state
Key-Bound Tokens	Preferred (bearer optional)	Always (no bearer)
Multiple Interactions	Redirect, user_code, app, push	Subscription + per-request
Request Continuation	Continuation access tokens	Subscription ID
Token Management	Rotate/revoke API	Optional management
Subject Information	Subject Identifiers, assertions	Resource owner info
Flexible Access Rights	Structured access descriptions	Scope with restrictions

5. Integration Possibilities

Can GAuth Use GNAP?

Conceptually Possible: GAuth could adopt GNAP's interaction modes and token management while retaining its PoA/compliance layer.

Potential Integration:

- GAuth Legal/Compliance Logic
- Power of Attorney
- Authorization Chains
- Commercial Register

GNAP Authorization Protocol

- Grant Negotiation
- Interaction Modes
- Token Management

Integration Steps:

1. **Map GAuth Subscription to GNAP Client Registration**
 - Use GNAP's dynamic instance_id
 - Store PoA credentials with instance
2. **Use GNAP Interaction Modes**
 - Leverage redirect/user_code for RO consent
 - Add PoA display during interaction
3. **Extend GNAP Access Token**
 - Add power_of_attorney field
 - Add authorization_chain field
 - Add compliance_level field
4. **Use GNAP Token Management**
 - Rotate Extended Tokens
 - Revoke based on PoA status

Example Hybrid Token:

```
{
  // GNAP fields
  "value": "gauth_gnap_token...",
  "access": ["financial_api"],
  "expires_in": 3600,
  "manage": {...},

  // GAuth extensions
  "power_of_attorney": {
    "poa_id": "poa_xyz",
    "issuer": "Owner's Authorizer"
  },
  "authorization_chain": [...],
  "compliance_level": "rfc-0111-compliant"
}
```

6. When to Use Which

Use GNAP (RFC 9635) When:

Building a **modern authorization system** to replace OAuth 2.0
Need **flexible grant negotiation** between client and AS
Supporting **diverse interaction modes** (web, mobile, IoT, devices)
Want **per-instance key binding** instead of client_id
Need **token rotation** and management
Building for **general-purpose** APIs and services
Want **IETF standard** ecosystem compatibility

Examples: - Modern web/mobile applications - IoT device authorization - API platforms - Microservices authorization

Use GAuth (RFC 0111/0115) When:

Authorizing **AI agents** or autonomous systems
Legal power of attorney relationships must be modeled
Compliance with GDPR, HIPAA, sector regulations required
Authorization chains must be cryptographically proven
Need **commercial register** integration
AI governance and accountability are critical
Operating in **regulated industries** (finance, healthcare, legal)

Examples: - Healthcare AI with guardian authorization - Financial AI with board authority - Legal AI with power of attorney - Corporate AI with compliance requirements

Use Both Together When:

Building **modern authorization** for **AI systems with legal authority**
Want **GNAP's flexibility** + **GAuth's compliance**
Need **grant negotiation** + **authorization chains**
Regulatory **AI systems** needing **interoperability**

7. Technical Comparison Summary

Feature	GNAP	GAuth	Winner
Flexibility	High (Grant negotiation)	Medium (Subscription fixed)	GNAP
Legal Framework	None	Comprehensive (PoA, compliance)	GAuth
Interaction Modes	4+ modes	Subscription + request	GNAP
State Management	Explicit states	Implicit (subscription-based)	GNAP
Token Security	Optional bearer	Always bound	GAuth
Delegation Depth	Single-level	Multi-level chains	GAuth
Compliance Tracking	None	Built-in	GAuth
Standards Ecosystem	IETF	Gimel Foundation	GNAP
AI Governance	Not addressed	Core feature	GAuth
Commercial Register	Not addressed	Integrated	GAuth

8. Evolution and Timeline

OAuth 2.0 → GNAP → GAuth

OAuth 2.0 (2012)

↓

Limitations identified:

- Rigid flows
- Bearer tokens
- `client_id/client_secret`
- No grant negotiation

↓
 GNAP (2024)

- Grant negotiation
- Per-instance keys
- Multiple interaction modes
- Token management

↓
 GAuth (2025)

- Builds on OAuth 2.0 concepts
- Adds PoA framework
- Adds compliance layer
- Specialized for AI

Key Insight: GNAP is OAuth 2.0's **successor**, while GAuth is OAuth 2.0's **specialized extension**.

9. Code Examples

GNAP Request

```
POST /tx HTTP/1.1
Host: as.example.com
Content-Type: application/json
Signature-Input: sig1=...
Signature: sig1=...

{
  "access_token": {
    "access": ["photo-api"]
  },
  "client": {
    "key": {
      "proof": "httpsig",
      "jwk": {...}
    }
  },
  "interact": {
    "start": ["redirect"],
    "finish": {
      "method": "redirect",
      "uri": "https://client.example/callback",
      "nonce": "ABC123"
    }
  }
}
```

GAuth Request

```
POST /v1/token/rfc HTTP/1.1
Host: gauth.as.example.com
Content-Type: application/json
Signature: ...

{
  "subscription_id": "sub_abc123",
```

```
"requested_scope": {  
  "actions": ["read", "write"],  
  "resources": ["patient_records"]  
},  
"poa_credential_ref": "poa_xyz789"  
}
```

10. Conclusion

GNAP and GAuth serve different purposes:

- **GNAP (RFC 9635):** Modern, flexible authorization protocol designed to replace OAuth 2.0 for general-purpose use cases
- **GAuth (RFC 0111/0115):** Specialized authorization framework for AI systems with legal power of attorney and compliance requirements

They are NOT competing - GNAP provides technical authorization flexibility, while GAuth provides legal/compliance governance for AI.

Potential Synergy: GAuth could adopt GNAP's interaction modes and token management while maintaining its unique PoA and compliance features, creating a powerful hybrid for regulated AI systems.

References

- RFC 9635 - Grant Negotiation and Authorization Protocol (GNAP)
- RFC 6749 - OAuth 2.0
- GiFo-RFC 0111 - GAuth 1.0 Authorization Framework
- GAuth Gap Matrix
- GAuth Architecture
- RFC 9493 - Subject Identifiers for Security Event Tokens